

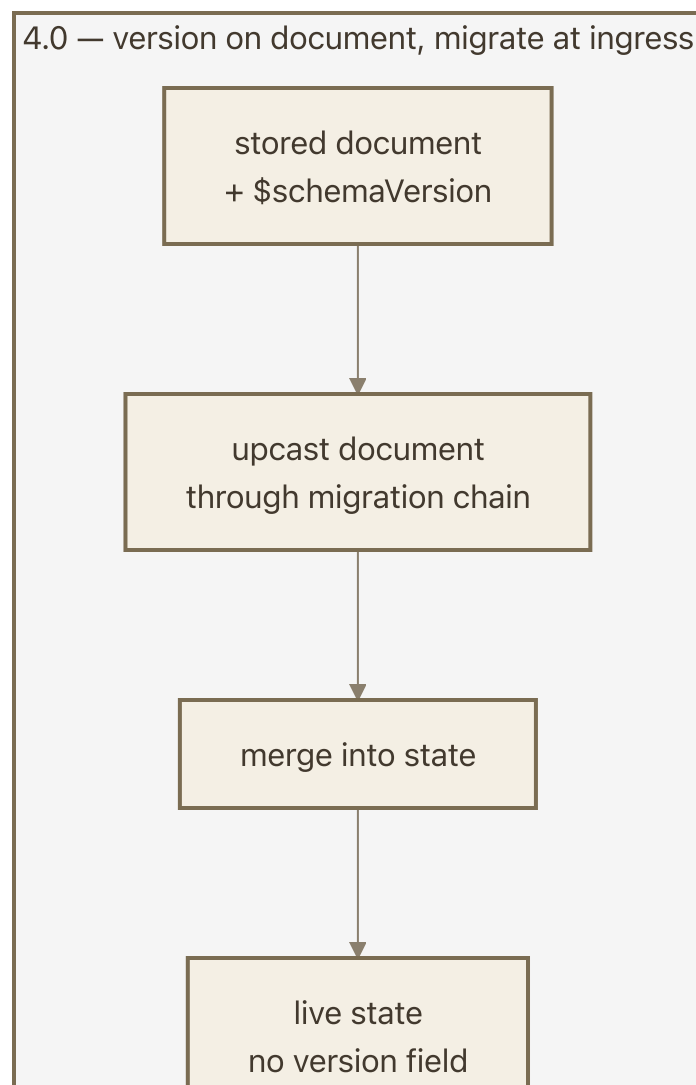
n-domain v4 — a guided walkthrough of the schema-migration redesign

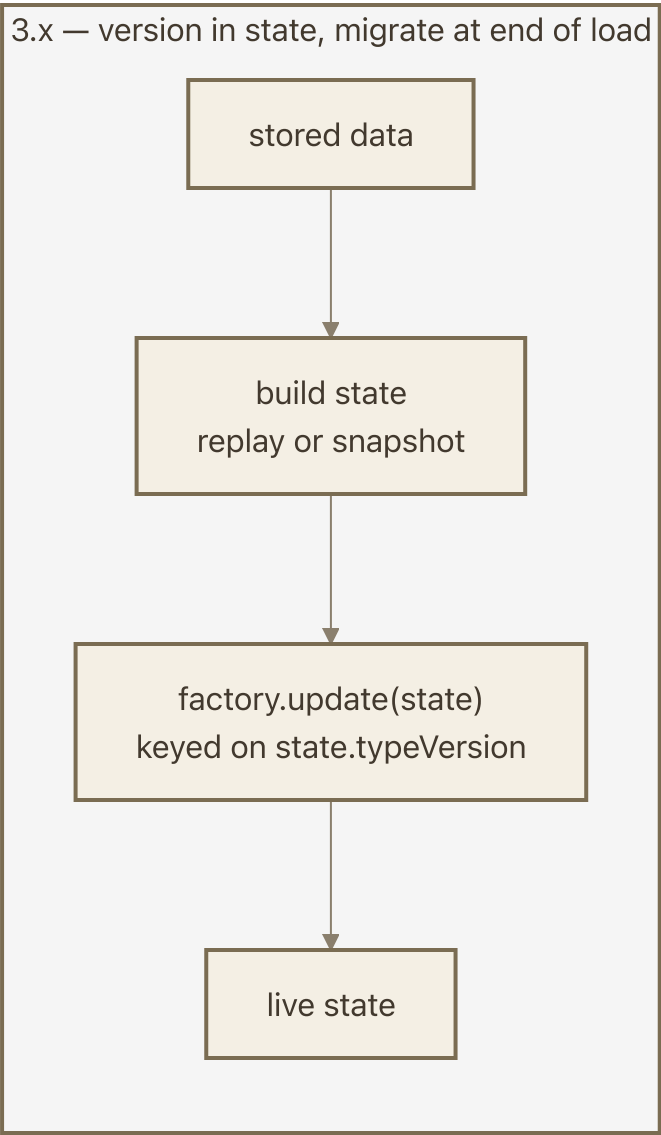
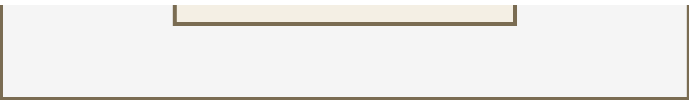
This explains every staged change, organized by **runtime flow** rather than by file, so you can follow what actually happens when an aggregate is created, replayed, snapshotted, and rebased. A file-by-file tour and a suggested diff-review order are at the end.

1. The change in one paragraph

In 3.x, the schema version (`typeVersion`) lived **inside live state**, and migration happened **once, at the end of loading**, via `factory.update()` . The investigation proved that model is only coherent for snapshots: replayed state is *computed* by current code, and after replay its fields have mixed origins (some from current-code `applyEvent` , some from historical overlay payloads), so it has no single "from version" — which is why the `typeVersion` guard was tautological on replay and why migrations silently never ran on streams.

v4 inverts this: **the version now lives on each stored document** (as `$schemaVersion` metadata), and **migration happens where each document enters the system** — before its data ever touches live state. Live state never carries a version at all.





Three kinds of stored state documents exist, and they are all the **same schema in the same encoding** (the output of `serializeStateIntoSnapshot`), so they share one migration seam:

Stored document	Written when	Read when
Snapshot	<code>snapshot()</code>	<code>deserializeFromSnapshot</code>
Frozen default state (<code>\$frozenDefaultState</code> on the created event)	aggregate creation	start of every replay
Rebase baseline (<code>\$baseline</code> on a rebase event)	<code>rebase()</code>	mid-replay, when the rebase event applies

2. The new vocabulary (4 small pieces)

`BASE_STATE_KEYS` ([aggregate-state.ts](#)) — the six framework-owned fields (`id`, `version`, `createdAt`, `updatedAt`, `isRebased`, `rebasedFromVersion`). `typeVersion` is gone from the interface. Everything that is not a base key is a *domain key*. `clearBaseState` still deletes a `typeVersion` key if one shows up (hygiene for old artifacts).

`StateMigration` ([state-migration.ts](#)) — one step: `migrate(payload) => payload`. A pure transform on the **serialized** domain-key payload of a stored document, moving it one version forward. Steps get a private deep copy, so mutating in place is fine.

`schemaVersion` is **derived, never declared** ([aggregate-state-factory.ts](#)):

```
public get schemaVersion(): number { return this.defineMigrations().length + 1; }
```

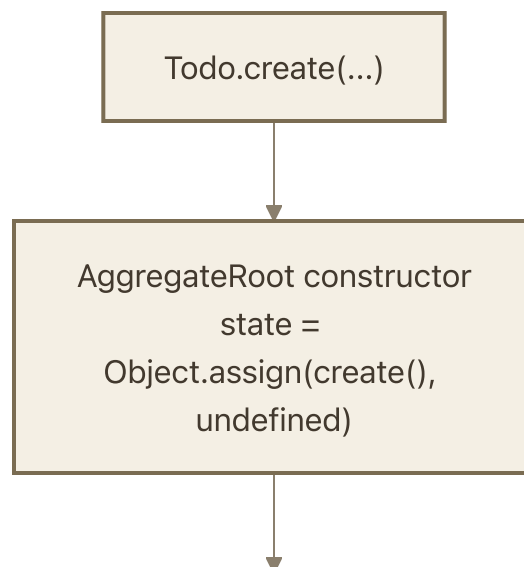
No migrations → version 1. One migration → version 2. You *cannot* bump the version without writing the step, and you cannot write a step without bumping the version. This replaces `typeVersion: N` in `create()` plus the honor-system `update()`.

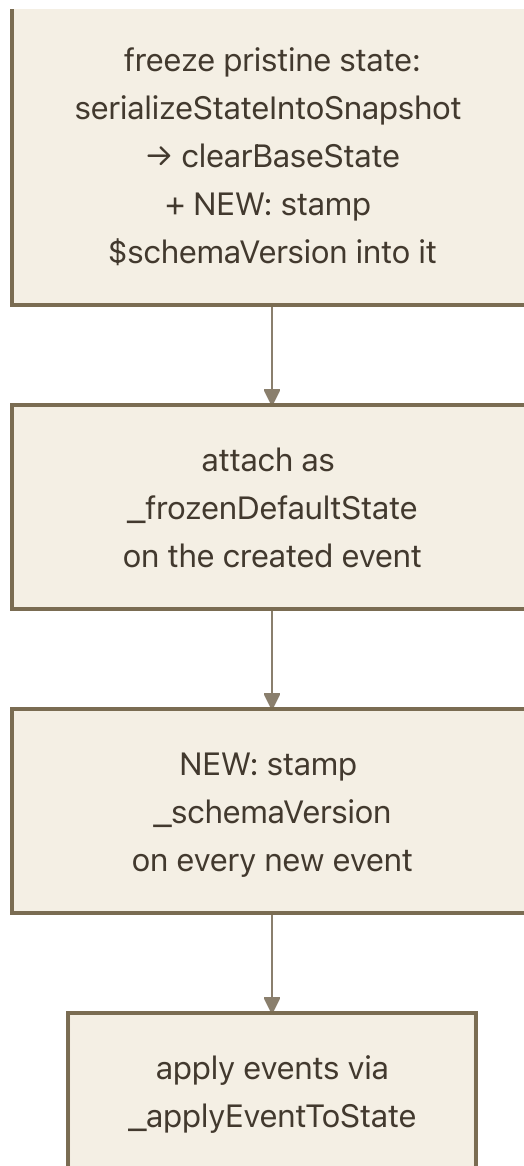
The upcast seam — `upcastStateDocument(payload, declaredVersion, kind)`:

1. rejects documents claiming a version newer than the code,
2. deep-clones the payload (`structuredClone`) so migration steps can never touch the caller's stored object,
3. folds the migration steps from `declaredVersion` up to current,
4. **conformance guard**: any key left that doesn't exist on the current `create()` output → throw (that's a rename/zombie you forgot to migrate). Missing keys are fine — they fall through to `create()` defaults, which is why *adding* a field never needs a step.

3. The five runtime flows

Flow A — creating a new aggregate (unchanged in spirit, two stamps added)

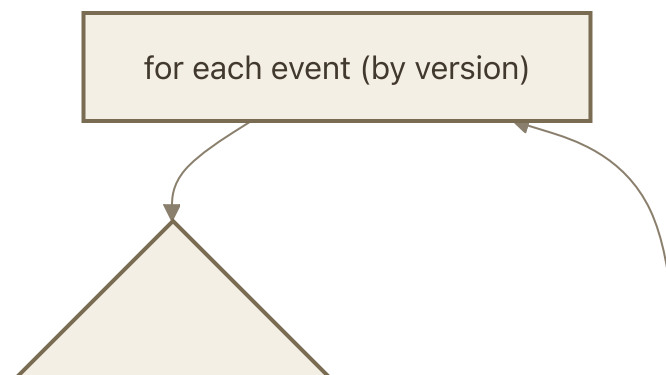


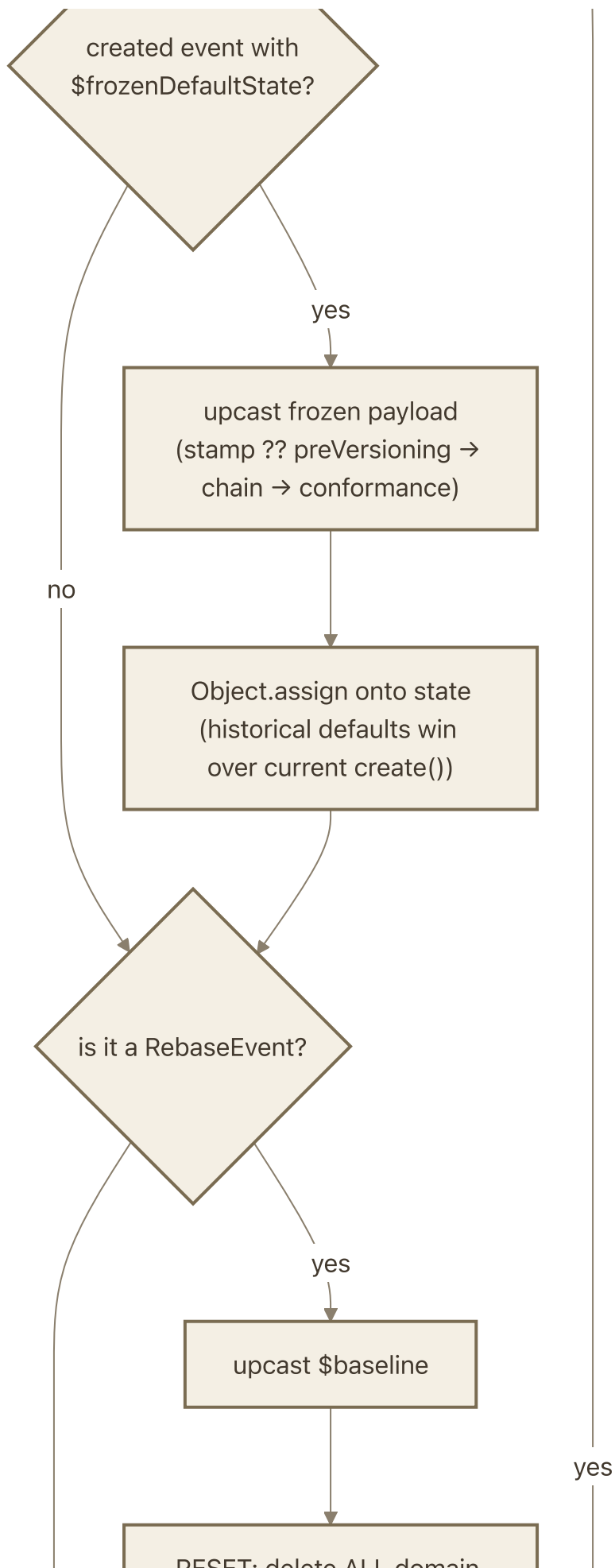


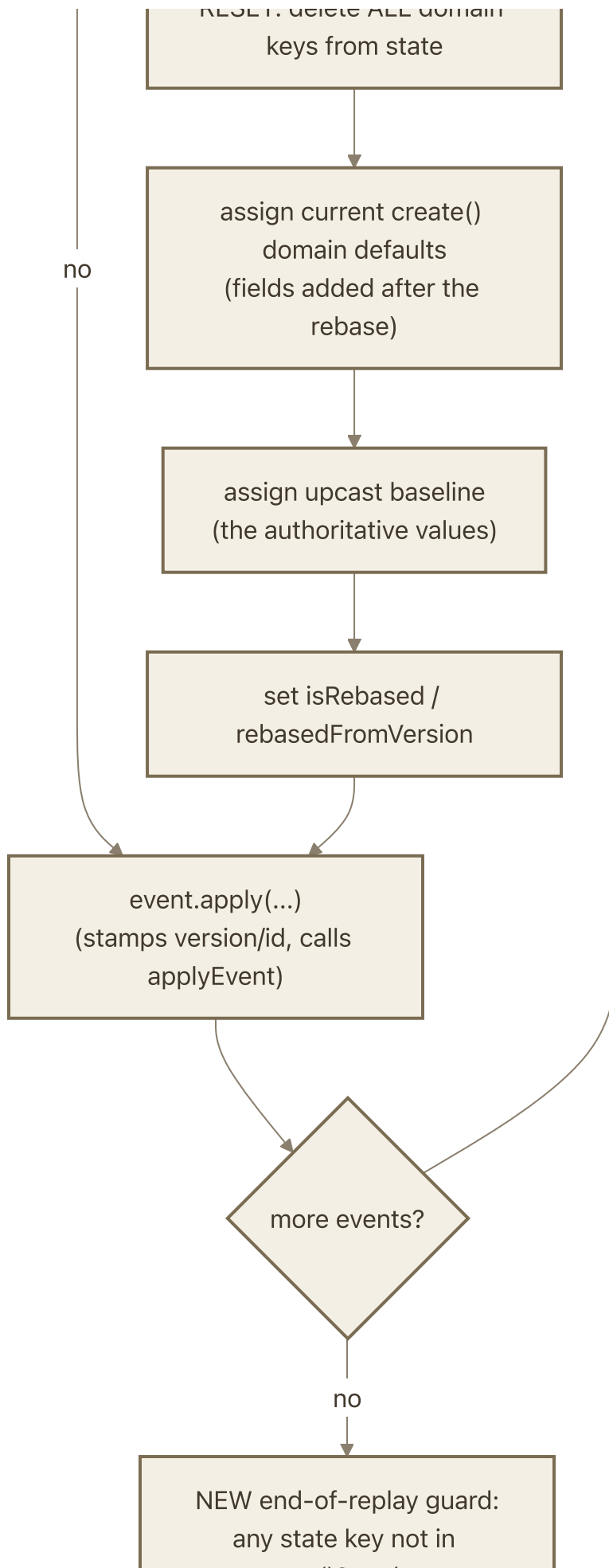
Same freeze mechanism you built in fb05793 — the only additions are the two write-time stamps, so that in the future every stored artifact knows which shape it was written in.

Flow B — loading from events (replay). This is the heart of the diff.

The constructor's replay loop no longer calls `event.apply(...)` directly. Every event — retro replay *and* new events via `applyEvent()` — now goes through one private seam, `_applyEventToState`, because that seam has what `DomainEvent.apply()` never had: **access to the state factory**, which is needed to run migrations.





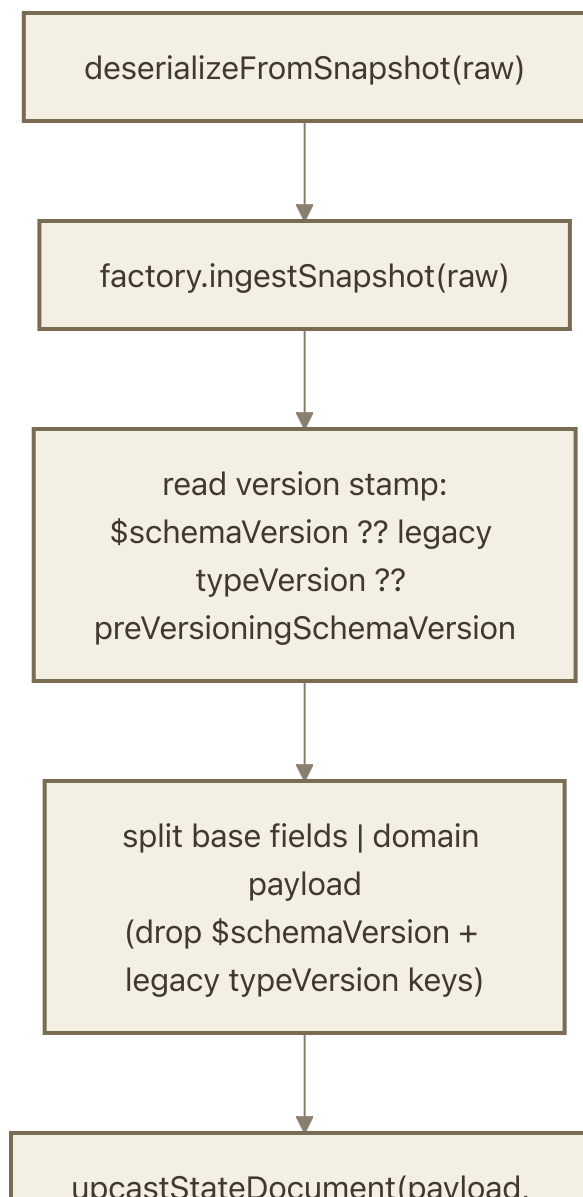


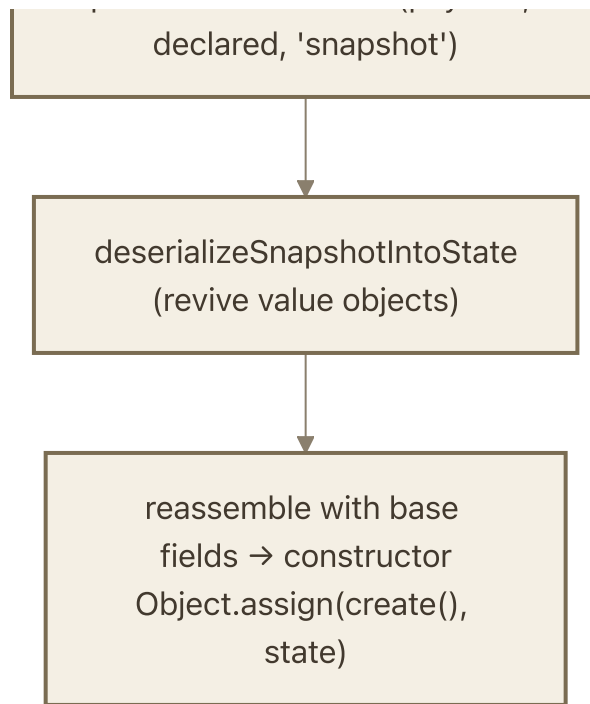
`create(): → throw`

Three things to notice:

- **The frozen-default overlay moved out of** `DomainEvent.apply()` into this seam. Same position in the flow (before the created event's own `applyEvent()`), but now the payload is *upcast first*, so a stream frozen in an old shape replays correctly after you rename a field.
- **Rebase overlay semantics changed from MERGE to RESET.** The old `Object.assign(state, defaultState, rebaseState)` could only *add/overwrite* keys — it could never remove a stale key or reset a renamed field, which is exactly how zombie keys survived. RESET wipes all domain keys first, so nothing before the rebase can leak past it.
- **The end-of-replay guard** is the safety net for the one path that legitimately bypasses the migration chain: pre-4.0 rebase events (see Flow F). If any old-shape residue reached live state, loading fails loudly with a message naming the keys and the fix — instead of the 3.x behavior, which was silently wrong values.

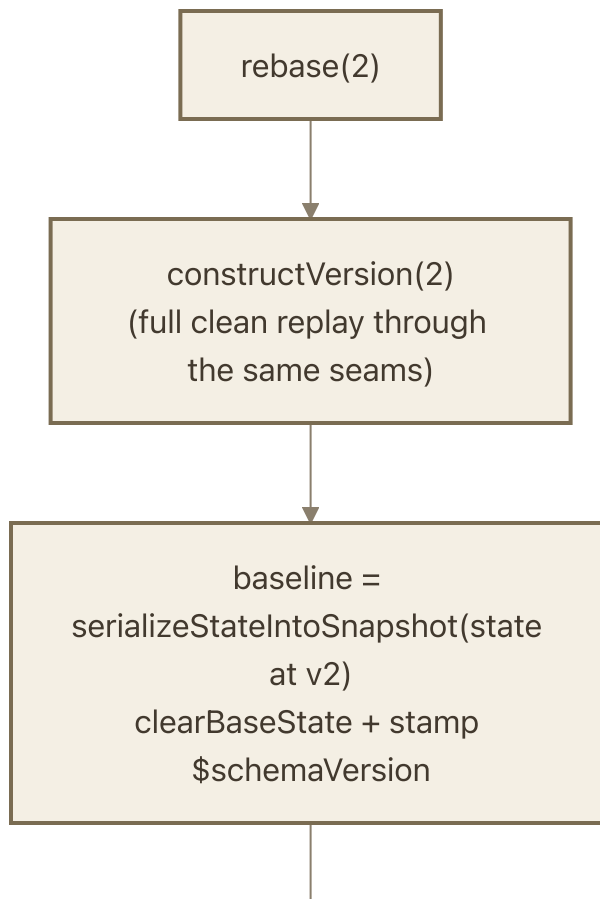
Flow C — loading from a snapshot

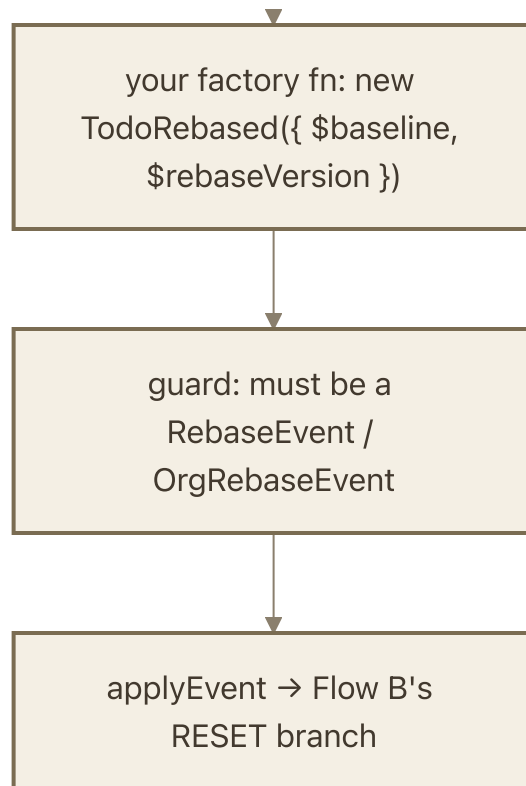




This replaces both `factory.deserializeSnapshot()` (the mechanical revival, now internal) and the snapshot half of `update()` (the migration, now a chain step). Old 3.x snapshots work because their in-band `typeVersion` is accepted as the stamp — it was the one honest version stamp the old system had. A legacy `typeVersion` larger than your chain throws a dedicated "port the pre-4.0 `update()` chain" error rather than a misleading one.

Flow D — calling `rebase(version)`





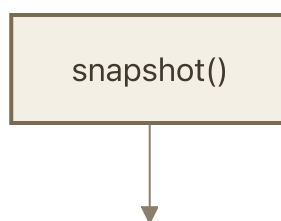
What changed: the old API gave your factory function **two** payloads (`defaultState` , `rebaseState`) and *your event class* applied them via `AggregateStateHelper.rebaseState` . Now the framework produces **one** baseline (the `rebaseState` was already the complete state; the separate `defaultState` became redundant once RESET assigns current `create()` defaults), and applies it itself. Your rebase event class shrinks to pure identity:

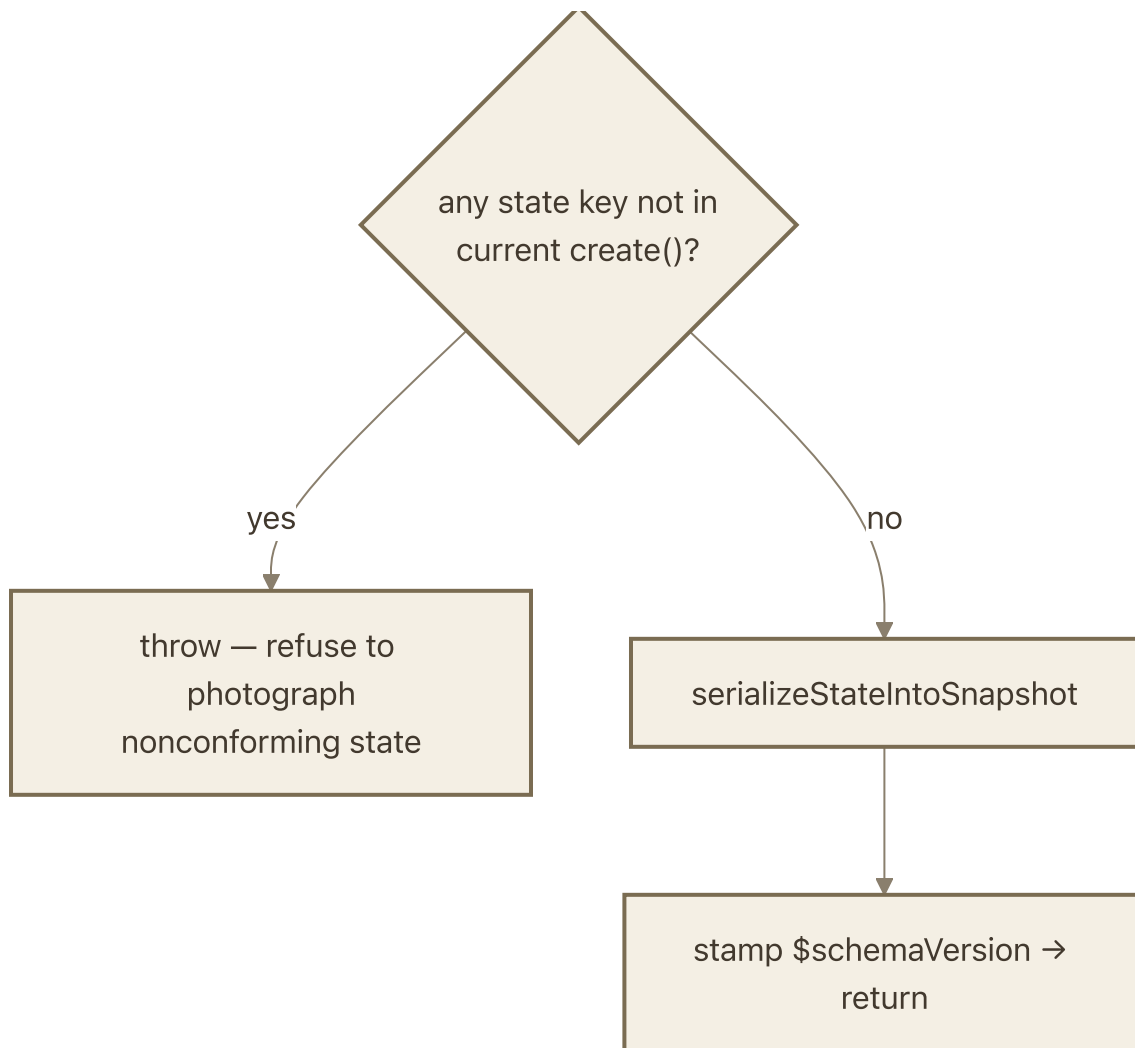
```
@serialize("Test")
export class TodoRebased extends RebaseEvent<TodoState>
{
    public get refType(): string { return "Todo"; }
}
```

`applyEvent` on rebase events is a framework-enforced no-op — the constructor throws if a subclass tries to override it. Because a rebase baseline is written at the **current** schema version, rebasing a stream is also how you stop paying the upcast chain on it (and how you physically shed an old shape).

`OrgRebaseEvent` exists because org aggregates couldn't use `RebaseEvent` : their `applyEvent` guard requires `OrgDomainEvent` , and single inheritance can't satisfy both. It's the same class re-based onto `OrgDomainEvent` (so it inherits `$organizationId` stamping), and the two are recognized at the framework seams via a shared brand symbol (`isRebaseEvent`) instead of `instanceof` .

Flow E — taking a snapshot()





The egress guard closes the worst 3.x hole: previously, replayed state carrying old-shape residue could be snapshotted **stamped with the current typeVersion**, permanently laundering the residue past the one path where migration worked. Now that snapshot is impossible to produce.

Flow F — legacy data (what happens to everything 3.x wrote)

3.x artifact	v4 behavior
Snapshot (typeVersion in-band)	loads; in-band value used as the version stamp, then dropped
Event without \$schemaVersion	replays unchanged; never retro-stamped ; re-serializes byte-identical (pinned by test)
Created event without frozen default (pre-fb05793)	overlay skipped, exactly as before
Created event with unstamped frozen default	treated as version 1 (preVersioningSchemaVersion , overridable) — exact for your estate since typeVersion never exceeded 1

Old user-defined rebase event	applies through the deprecated <code>AggregateStateHelper.rebaseState</code> merge path, behavior-identical — <i>until</i> you ship a migration step, at which point old-shape payloads hit the end-of-replay guard loudly. Remediation: re-rebase those streams first
New artifacts read by 3.x code	not supported — rollback is one-way once v4 writes snapshots/rebases

4. File-by-file tour (suggested diff-review order)

1. [src/aggregate-state.ts](#) — `typeVersion` deleted; `BASE_STATE_KEYS` added; `clearBaseState` driven by it.
2. [src/state-migration.ts](#) (*new*) — the `StateMigration` step contract.
3. [src/aggregate-state-factory.ts](#) — deletions: `update()`, `public deserializeSnapshot()`, `typeVersion` in `createDefaultAggregateState()`. Additions: `derived schemaVersion`, `defineMigrations()`, `preVersioningSchemaVersion`, the upcast seam (`upcastStateDocument`), the snapshot ingress (`ingestSnapshot`), and `_fetchMigrations()` (validates the user chain once, so the derived version and the fold can never disagree).
4. [src/rebase-event.ts](#) (*new*) — `RebaseEvent` + `RebaseEventData` + the brand (`isRebaseEvent`) + the no-override constructor guard.
5. [src/org-rebase-event.ts](#) (*new*) — same contract on the `OrgDomainEvent` trunk.
6. [src/domain-event.ts](#) / [domain-event-data.ts](#) — `_schemaVersion / $schemaVersion` (serialized only when set, so legacy events stay byte-identical); the frozen-default overlay **removed** from `apply()` (a comment marks where it went).
7. [src/aggregate-root.ts](#) — the heart:
 - constructor: `update()` call + `typeVersion` guard deleted; mixed-stream guard (all events stored or all new — prevents retro-stamping); freeze block stamps `$schemaVersion`; replay loops route through `_applyEventToState`; end-of-replay conformance guard.
 - `_applyEventToState` / `_upcastArtifact` / `_findNonConformingKeys` — Flow B above.
 - `snapshot()` — egress guard + stamp. `rebase()` — Flow D. `deserializeFromSnapshot` → `ingestSnapshot`. `applyEvent()` — stamps new events, routes through the seam.
8. [src/aggregate-state-helper.ts](#) — `rebaseState()` kept but `@deprecated` (legacy apply path only); `describeShape()` added (sorted key list + fingerprint) for the manifest drift guard.
9. [src/index.ts](#) — exports: `BASE_STATE_KEYS`, `StateMigration`, `StateArtifactKind`, `RebaseEvent(Data)`, `OrgRebaseEvent(Data)`.
10. **Tests** — fixtures: `MigratedTodoStateFactory` is now a real chain example (`v1 legacyTitle` → `v2 title`); `TodoRebased` re-parented; `LegacyTodoRebased` replicates the pre-4.0 event to pin the legacy path; `org-todo.ts` is the first org fixture domain. Suites: the old "Type version" tests became the "Schema migration" suite (each ingress migrated + guarded, RESET semantics, byte-identical legacy re-serialization, stamping, mixed-stream, override guard); `org-domain.test.ts` pins org rebase round-trip; the fingerprint test became the shape-manifest test (`state-shapes.json` — key

removals demand a migration step, value drift demands an acknowledgment).

11. **README** — migrations/rebase/API sections rewritten; the false "stream fails fast" claim replaced with the true mechanics; "Upgrading from 3.x" added. **package.json** — 4.0.0 (note: don't run `publish-package` as-is; its `yarn version patch` would ship 4.0.1).
-

5. The guard inventory (what fails loudly now, and where)

Guard	Where	Catches
Ingress conformance (extra keys)	<code>upcastStateDocument</code>	renamed/removed keys with no migration step; zombie keys
Newer-artifact	<code>upcastStateDocument</code>	document written by newer code
Under-ported legacy chain	<code>ingestSnapshot</code>	3.x <code>typeVersion</code> beyond your <code>defineMigrations()</code>
Chain validity	<code>_fetchMigrations</code>	malformed <code>defineMigrations()</code>
Egress conformance	<code>snapshot()</code>	nonconforming state being photographed (anti-laundering)
End-of-replay conformance	constructor (replay branch)	old-shape residue from pre-4.0 rebase payloads
Rebase event type	<code>rebase()</code>	rebase events not extending <code>RebaseEvent / OrgRebaseEvent</code>
No-override	<code>RebaseEvent / OrgRebaseEvent</code> ctor	subclasses overriding <code>applyEvent</code>
Mixed streams	constructor	stored + new events mixed (would retro-stamp legacy events)